Casos Posibles que el Profesor Podria Pedir en Vivo

Ademas del flujo basico (bloquear -> confirmar), el profesor puede pedirte que demuestres estos escenarios. Cada uno tiene el comando exacto, lo que deberia salir, y que decir.

Caso 1: Intentar bloquear un asiento que ya esta reservado

```
Invoke-RestMethod -Uri "http://localhost:3000/events/evt-001/seats/A1-1/hold" -Method POST -ContentType
"application/json" -Body '{"customerId":"cust-999"}'
```

Esperado: Error 409: {"error":"Seat is booked"} (un error rojo en PowerShell aqui es lo correcto, no un bug)

> "Esto demuestra que el sistema no permite reservar dos veces el mismo asiento: valida el estado antes de crear el bloqueo."

Caso 2: Confirmar un asiento sin haberlo bloqueado antes

```
Invoke-RestMethod -Uri "http://localhost:3000/events/evt-001/seats/A3-4/confirm" -Method POST
-ContentType "application/json" -Body '{"customerId":"cust-777"}'
```

Esperado: Error 410: {"error":"Hold expired or does not exist"}

> "El sistema exige que primero exista un bloqueo activo antes de poder confirmar; evita reservas sin pasar por el flujo correcto."

Caso 3: Confirmar con un cliente distinto al que hizo el bloqueo

```
Paso A (bloquear con cust-111):
Invoke-RestMethod -Uri "http://localhost:3000/events/evt-001/seats/A2-5/hold" -Method POST -ContentType
"application/json" -Body '{"customerId":"cust-111"}'

Paso B (confirmar con cust-222, OTRO cliente):
Invoke-RestMethod -Uri "http://localhost:3000/events/evt-001/seats/A2-5/confirm" -Method POST
-ContentType "application/json" -Body '{"customerId":"cust-222"}'
```

Esperado: Error 403: {"error":"Hold belongs to another customer"}

> "El bloqueo le pertenece a quien lo creo; otro cliente no puede confirmarlo aunque sepa el ID del asiento. Esto evita que alguien le robe la reservacion a otra persona."

Caso 4: Lista de espera (Sorted Set)

```
Agregar:
Invoke-RestMethod -Uri "http://localhost:3000/events/evt-001/waitlist" -Method POST -ContentType
"application/json" -Body '{"customerId":"cust-300"}'

Ver la lista (ordenada por tiempo de solicitud):
Invoke-RestMethod -Uri "http://localhost:3000/events/evt-001/waitlist"
```

Esperado: Regresa la lista de clientes en orden de llegada, con su "score" (timestamp)

> "Uso un Sorted Set de Redis para que la lista de espera quede ordenada automaticamente por el momento en que cada cliente se registro, sin que yo tenga que ordenarla manualmente."

Caso 5: Actualizar datos del evento

```
Invoke-RestMethod -Uri "http://localhost:3000/events/evt-001" -Method PUT -ContentType "application/json"
-Body '{"price":"500"}'
```

Esperado: {"id":"evt-001","updated":["price"]}

"Esto actualiza solo el campo que mande, sin tocar los demas datos del evento (operacion UPDATE del CRUD)."

Caso 6: Cancelar una reservacion ya confirmada

```
Usa el bookingId real que te devolvio tu propio /confirm anterior (se ve en la respuesta de ese paso), por ejemplo:
Invoke-RestMethod -Uri "http://localhost:3000/bookings/evt-001-A3-2-1793320000000" -Method DELETE
```

Esperado: {"cancelled":"","seatReleased":"A3-2"}

"Cancelar una reservacion (DELETE del CRUD) libera el asiento de vuelta a 'available' automaticamente."

Caso 7: Demostrar que los datos SI persisten en la nube (no en tu servidor)

```
Paso A: en la Terminal 1, presiona Ctrl+C para apagar el servidor.
Paso B: corre otra vez npm start.
Paso C: vuelve a correr Invoke-RestMethod -Uri "http://localhost:3000/events/evt-001/seats"
```

Esperado: Los asientos que ya estaban "booked" o "held" (si no vencio el TTL) siguen igual, no se resetean solos

"Aunque apague y prenda mi servidor, los datos siguen ahi porque viven en Redis Cloud, no en la memoria de mi aplicacion. Eso es lo que hace que sea una base de datos de verdad y no solo una variable temporal."

Tip general: si el profesor pide algo que no esta en esta lista, recuerda la idea central: todo gira alrededor del bloqueo temporal con TTL (SET NX EX) y de que los datos persistentes (eventos, asientos, reservaciones confirmadas) no tienen TTL. Con esas dos ideas puedes razonar cualquier pregunta nueva en el momento.